

1. Syntax of C and C++

C Programming language

A procedural programming language used for system and application development.

C++ Programming language

An object-oriented programming language developed as an extension of C.

Structure

1) Documentation Section

Contains comments describing the program.

2) Header Files

Include library functions required in the program.

3) Global Variables

Variables declared outside all functions.

9) main() Function

The starting point of program execution.

10) Declaration Section

Used to declare variables before use.

11) Executable Section

Contains instructions executed by the compiler.

12) Return Statement

Ends the function and returns a value.

```
/* Documentation Section  
Program to demonstrate the structure  
of a C program */
```

```
#include <stdio.h> // Header File Section
```

```
int globalVar = 100; // Global Variable Section
```

```
int main() // Main Function
```

```
{  
    // Declaration Section
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    int sum;
```

```
    // Executable Section
```

```
    sum = a + b;
```

```
    printf("Value of a = %.d\n", a);
```

```
    printf("Value of b = %.d\n", b);
```

```
    printf("Sum = %.d\n", sum);
```

```
    printf("Global Variable = %.d\n", globalVar);
```

```
    // Return Statement
```

```
    return 0;
```

```
}
```

Compile Result

Value of a = 10

Value of b = 20

Sum = 30

Global Variable = 100

[Process completed - press Enter]

Tokens

Tokens

Smallest individual units of a program

Keywords

Reserved words with predefined meaning.

Identifiers

Names given to variables, functions, and arrays.

Constants

Fixed values that do not change during execution.

Strings

Sequence of characters enclosed in double quotes.

Operators

Symbols used to perform operations.

SUNDAY 07

Special Symbols

Characters with special meanings like `{ }`, `;`, `( )`.

Variables

A named memory location used to store data.

Declaration

Specifying the variable name and type.

Initialization

Assigning an initial value to a variable.

Input And Output Functions:-

Input Functions

Used to take data from the users.

scanf()

Reads input from the keyboard.

Output Function

Used to display data on the screen.

1

printf()

Prints output to the screen.

* Comments

Non executable statements used to explain codes.

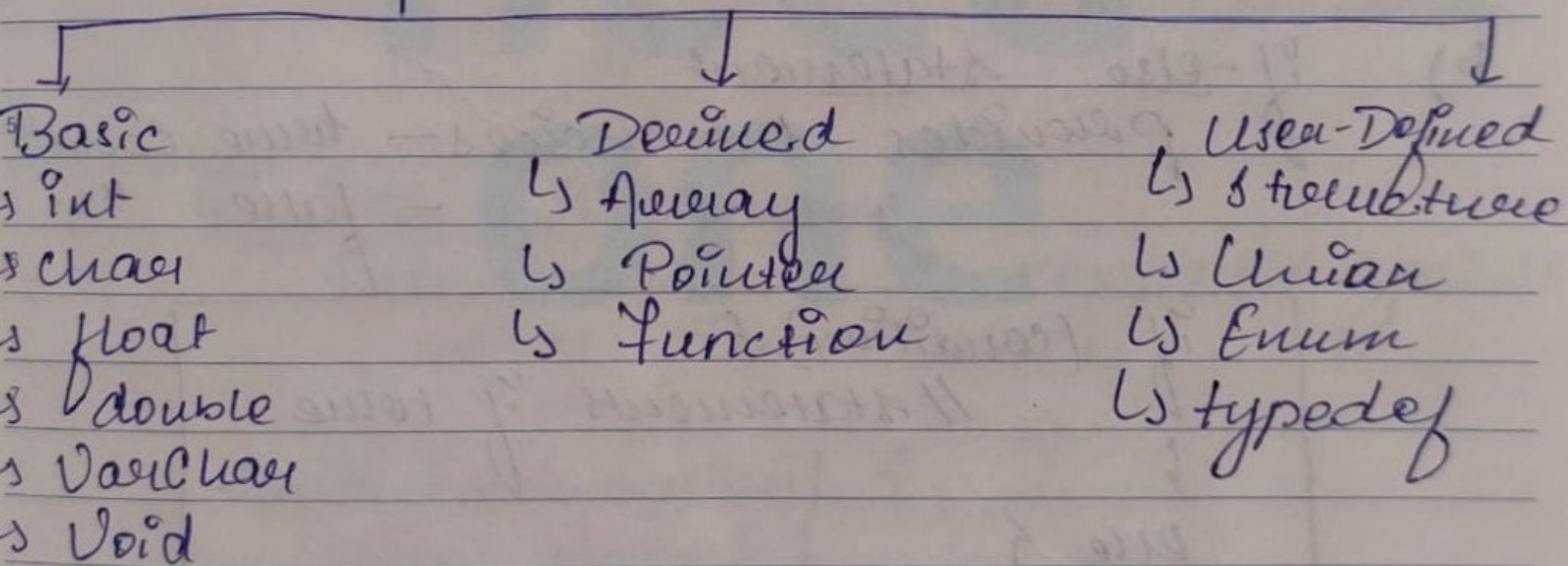
Single line

Begins with //.

Multi line

Enclosed with /* */.

Data Types.



2. Conditions And loops :-

Conditions

Allow a program to make decisions based on whether an expression is true or false.

- a) if statement
executes a block of code only when the condⁿ is true.

```
if (condition) {  
    // statements  
}
```

- b) if-else statement
provides two choices - true condⁿ
- false "

```
if (condition) {  
    // statements if true  
}  
else {  
    // statements if false  
}
```

c) else - if ladder
multiple condⁿ to check.

```
if (condition 1) {  
    // code  
}  
else if (condition 2) {  
    // code  
}  
else {  
    // code  
}
```

d) Nested - if Statement
if " inside another
" "

e) Switch Statement

One variable is compared against many fixed values.

```
switch (expression) {  
    case value 1 :  
        // code  
        break ;  
    case value 2 :  
        // code  
        break ;  
    default :  
        // code
```

Loops (Iteration)

Loops execute a block of code repeatedly until a condition becomes false.

a) for loop
When no. of loops is known.

```
for (initialization; condition; update) {  
    // statements  
}
```

b) while loop

Checks the condition before executing the loop body.

```
while (condition) {  
    // statements  
}
```

c) do-while loop

executes the loop body atleast once because the condⁿ is checked after execution.

```
do {  
    // statements  
} while (condition);
```

Loop Control Statements

a) break

Terminates the loop immediately.

b) continue

Skips the current iteration and moves to the next one.

PATTERN PROGRAMS

1. Square Pattern

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, j;
```

```
    for (i = 1; i <= 5; i++)
```

```
    {
```

```
        printf ("*");
```

```
    }
```

```
    printf ("\n");
```

```
    return 0;
```

```
}
```

SUN

Compile Result

```
*****  
*****  
*****  
*****  
*****
```

```
[Process completed - press Enter]
```

2. Hollow Square

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, j, n = 5;
```

```
    for (i = 1; i <= n; i++)
```

```
    {
```

```
        if (i == 1 || i == n || j == 1 || j == n)
```

```
            printf("*");
```

```
        else
```

```
            printf(" ");
```

```
    }
    printf("\n");
```

```
    return 0;
```

```
}
```

Compile Result

```
*****
```

```
*      *
```

```
*      *
```

```
*      *
```

```
*****
```

```
[Process completed - press Enter]
```

3. Triangle

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, j;
```

```
    for (i = 1; i <= 5; i++)
```

```
    {
```

```
        for (j = 1; j <= i; j++)
```

```
        {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Compile Result

```
*  
**  
***  
****  
*****
```

```
[Process completed - press Enter]
```

4.) Inverted Triangle

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i, j;
```

```
    for (i = 5; i >= 1; i--)
```

```
    {
```

```
        for (j = 1; j <= i; j++)
```

```
        {
```

```
            printf("*");
```

```
        }
        printf("\n");
```

```
    }
    return 0;
```

Compile Result

**

*

[Process completed - press Enter]

5) Pyramid

```
#include <stdio.h>
```

```
int main()  
{  
    int i, j, k, n = 5;
```

```
    for (i = 1; i <= n; i++)
```

```
    {  
        for (j = 1; j <= n - i; j++)  
            printf(" ");
```

```
        for (k = 1; k <= 2 * i - 1; k++)  
            printf("* ");
```

```
        printf("\n");
```

```
    }  
    return 0;  
}
```

Compile Result

```
    *  
  ***  
*****  
*****  
*****
```

[Process completed - press Enter]

6) Inverted Pyramid

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int i, j, k, n = 5;
```

```
for (i = n; i > 1; i--)
```

```
for (j = 1; j <= n - i; j++)  
printf(" ");
```

```
for (k = 1; k <= 2 * i - 1; k++)  
printf("*");
```

```
printf("\n");
```

```
}  
return 0;
```

Compile Result

```
*****
```

```
*****
```

```
*****
```

```
***
```

```
*
```

```
[Process completed - press Enter]
```

7) Hay Diamond

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int i, j;
```

```
    for (i = 1; i <= 5; i++)
```

```
    {
        for (j = 1; j <= i; j++)
            printf ("*");
```

```
        printf ("\n");
```

```
    for (i = 4; i >= 1; i--)
```

```
    {
        for (j = 1; j <= i; j++)
            printf ("*");
```

```
        printf ("\n");
```

```
    return 0;
```

Compile Result

```
*  
**  
***  
****  
*****  
*****  
***  
**  
*
```

[Process completed - press Enter]

8) Diamond

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int i, j, k, n = 5;
```

```
    for (i = 1; i <= n; i++)
```

```
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
    }
```

```
    for (k = 1; k <= 2 * i - 1; k++)
        printf("* ");
```

```
    printf("\n");
```

```
    for (i = n - 1; i >= 1; i--)
```

```
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");
    }
```

```
    for (k = 1; k <= 2 * i - 1; k++)
        printf("* ");
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

Compile Result

```
  *  
 ***  
*****  
*****  
*****  
*****  
*****  
  *  
 ***  
  *
```

[Process completed - press Enter]